
Cypher Brain × ENSv2

Scoped Brain — Give every AI agent only the part of your brain it needs.

ETHGlobal Tokyo 2026 · ENS Prize · Continuity Track · Track 2: Best Integration of ENSv2 into an Existing Project



What this builds on | not a new project, an extension of existing OSS

We added ENSv2-based permission control to [Cypher Brain](#), an existing open-source project.



What Cypher Brain is

An already-running OSS project

- A CLI tool that encrypts and protects a personal "second brain" (gbrain)
- MIT license, single maintainer, a mature CI / release pipeline
- This submission adds new features to that existing project

A genuine extension of Cypher Brain, matching Track 2's "integration into an existing project" requirement

What is gbrain? | a personal knowledge base that updates every day

gbrain is my own "second brain." It records my conversations, decisions, and what I learn, automatically.

1

Daily record

Conversations, decisions, and insights accumulate automatically, every day.

This is my own personal knowledge base, not a shared tool

2

Always up to date

A nightly job refreshes it automatically.
No human updates it by hand.

The update itself is what an AI agent handles
(next slide)

3

Sensitive personal data

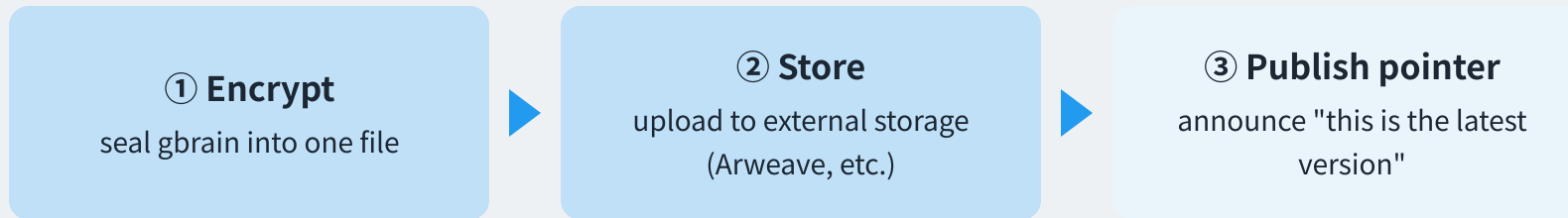
Who can access it, and how much, must be tightly controlled.

This is exactly the problem ENSv2 solves in this project

Protecting "personal data that updates itself every night" is the starting point for everything that follows

What Cypher Brain already does | encrypt, store, publish a pointer

Every night, these three steps run automatically to protect gbrain's contents.



Who actually does this?

Every night, an **AI agent** — not a human — runs these three steps automatically. Step ③, "update the pointer," is the one the agent has to be trusted with.

That "trust" in step ③ is exactly where the problem in the next slide comes from

The problem we had | no good way to hand an agent this one job

There was no way to give an agent only the permission it needs, in a way anyone could check.

Approach	How it works	The problem
Option A Share the key	Give the agent the owner's own wallet key	The agent can now do anything you can do, including moving money
Option B Private database	Manage access with a central server and an API key	A server can go down or be censored, and outsiders cannot verify the key is really limited
Our solution	Use ENSv2's Enhanced Access Control	A permission limited to one field, provable by anyone

What was missing was not just "limiting" the permission, but making that limit verifiable by outsiders

Our solution | two wallets, one narrow permission

The owner uses their real wallet once, to grant a separate agent wallet permission for exactly one field.



Owner wallet

The real key. Used only once. Never saved to disk.

ens-setup

grants a permission
for one field



Agent wallet

Generated by cypher-brain. Handles all daily updates by itself.

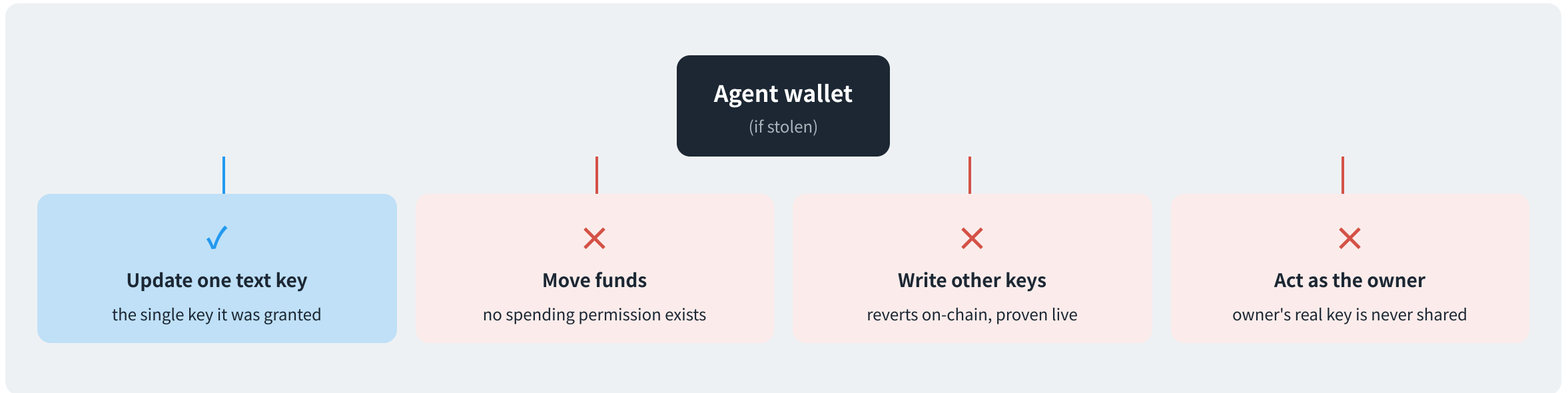
ens-verify: proving the permission is really narrow

Positive check — did the expected value actually get written? **Negative check** — does the same wallet really fail to write to a different field? Both together are what make this a proof, not a claim.

No custom smart contract was written -- this uses only what ENSv2 already provides

What this makes possible | even a stolen agent key is contained

If the agent wallet is ever compromised, here is everything it can, and cannot, do.



Not just a claim

This boundary is **verifiable by anyone** -- ens-verify's on-chain proof is public, not our word for it.

This proves the integration is functionally central, not a cosmetic add-on

The full architecture | three layers, each narrowing the scope

This submission builds layer three. Layers one and two already existed in Cypher Brain.



Layer three's permission model is per text-key, so it already generalizes to multiple brain slices (next slide)

Where this can go next | one permission foundation, many brain slices

Once gbrain can export per use case, today's permission model already supports splitting the brain by use case.

work-brain

for a work-only agent

can touch only this key

travel-brain

for a travel-booking agent

can touch only this key

private-brain

for the most trusted agent only

can touch only this key

independent grant

independent grant

independent grant

ENSv2 Enhanced Access Control (the permission foundation built in this Epic)

any number of independent per-key permissions, using only what already exists today

Splitting the brain by use case only needs work on gbrain's side -- the permission mechanism is already general-purpose

The live demo | every step run for real, on Sepolia testnet

Not a simulation -- this is what actually happened on-chain.

① Register ENS name

masashi-ono0611.eth



② Grant permission

ens-setup (owner key)



③ Write pointer

ens-set-text (agent key only)



④ Verify

ens-verify (positive +
negative PASS)

Repeated with three independent keys

masa-brain (the brain pointer), **agent-context** (self-description), and **agent-endpoint[mcp]** (server address) -- each one independently granted, written, and verified. Every transaction hash is listed in the written submission.

Proven live, on real Sepolia transactions, that the permission model actually works

Thank you

**Give your AI only what it needs.
Nothing more.**

github.com/Masashi-Ono0611/cypher-brain · Epic #972